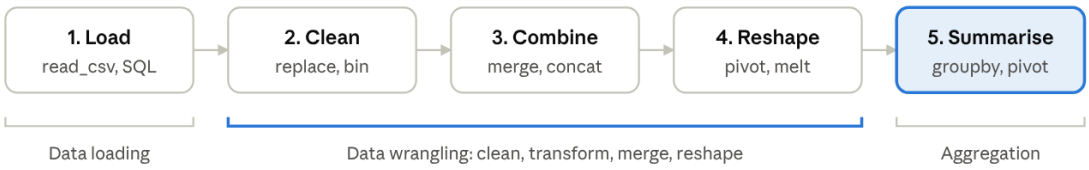
Data Science Unit 4: Data Preprocessing

Unit overview

Part	Syllabus topics	Typical job
Data loading, storage and file formats	Reading and writing text data, binary formats, HTML and web APIs, databases	Turn a CSV file, web page or SQL table into a DataFrame
Data wrangling: clean, transform, merge, reshape	Combining and merging data sets, reshaping and pivoting, data transformation, string manipulation	Join several tables, fix values, change a table's layout
Data aggregation and group operations	GroupBy mechanics, data aggregation, group operations and transformations, pivot tables and cross tabulation	Compute totals and averages per category

Preprocessing is a pipeline: load, clean, combine, reshape, summarise



preprocessing pipeline · 5 stages

1. Data Loading, Storage and File Formats

Before analysis, data must be read into a DataFrame; pandas can read and write text files, binary files, web pages, web APIs and databases, usually with one function call.

1.1 Reading and writing data in text format

`pd.read_csv` is the most used loader. It turns a delimited text file into a DataFrame, guessing column types for you. `pd.read_table` is the same function with a tab as the default separator.

Option	Purpose	Example
`sep`	Column separator	`sep=";"`
`header`	Row to use as column names	`header=None`

Option	Purpose	Example
	(`None` if there is none)	
`names`	Supply your own column names	`names=["id", "name"]`
`index_col`	Column to use as the row labels	`index_col="id"`
`usecols`	Load only some columns	`usecols=["id", "marks"]`
`dtype`	Force column types	`dtype={"id": str}`
`na_values`	Extra strings to treat as missing	`na_values=["NA", "-"]`
`parse_dates`	Convert columns to dates	`parse_dates=["joined"]`
`skiprows` / `nrows`	Skip lines at the top / read only the first n rows	`nrows=1000`
`encoding`	Text encoding of the file	`encoding="utf-8"`
`chunksize`	Read a big file in pieces	`chunksize=100_000`

```
import pandas as pd

df = pd.read_csv("students.csv", index_col="id",
                 parse_dates=["joined"], na_values=["NA", "-"])

df.to_csv("clean.csv", index=False)          # write; index=False leaves out
the row labels

total = 0
for chunk in pd.read_csv("big.csv", chunksize=100_000):   # each chunk is a
DataFrame
    total += chunk["amount"].sum()                        # process a piece
at a time

fixed = pd.read_fwf("report.txt")             # fixed-width columns
sheet = pd.read_excel("marks.xlsx", sheet_name="Sheet1")   # Excel (needs
openpyxl)
js = pd.read_json("data.json")                # JSON that looks like a table
```

When the file is JSON with nested structure, load it with Python's `json` module (`json.loads(text)`) and then build the DataFrame from the lists and dictionaries you want.

1.2 Binary data formats

Binary formats store data compactly and keep column types exactly, so they load faster than text.

Format	Write and read	Notes
CSV	`to_csv`, `read_csv`	Human-readable; every program understands it; types are lost

Format	Write and read	Notes
JSON	`to_json`, `read_json`	Readable; the standard format of web APIs
Excel	`to_excel`, `read_excel`	For sharing with spreadsheet users
pickle	`to_pickle`, `read_pickle`	Any Python object; load only files you trust
HDF5	`to_hdf`, `read_hdf`	Fast for large tables read in parts; needs PyTables
Parquet	`to_parquet`, `read_parquet`	Compressed, column-oriented, widely used for big data; needs pyarrow

```
df.to_pickle("students.pkl")
back = pd.read_pickle("students.pkl")

df.to_parquet("students.parquet")        # needs: pip install pyarrow
back = pd.read_parquet("students.parquet")
```

1.3 Interacting with HTML and web APIs

`pd.read_html(url)` finds every `<table>` in a web page and returns a **list** of DataFrames. For a web API, use the `requests` library to fetch the response, then turn the JSON into a DataFrame.

```
tables = pd.read_html("https://example.com/page-with-tables.html")   # needs
lxml
first = tables[0]

import requests

resp = requests.get("https://api.github.com/repos/pandas-dev/pandas/issues",
                    params={"per_page": 5})
resp.raise_for_status()                    # stop with an error for a bad
status code
data = resp.json()                         # a list of dictionaries
issues = pd.DataFrame(data, columns=["number", "title", "state"])
print(issues.head())
```

Check the status code, respect the site's rate limits and terms of use, and keep any API keys out of your code.

1.4 Interacting with databases

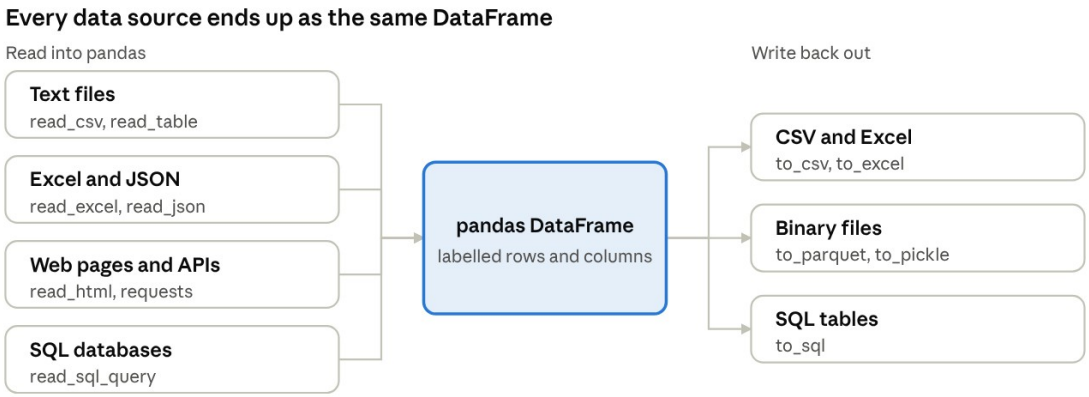
A database table maps naturally to a DataFrame. `read_sql_query` runs a SQL query and returns the rows; `to_sql` writes a DataFrame back as a table. SQLite needs only the standard `sqlite3` module; for MySQL or PostgreSQL create a connection through SQLAlchemy.

```
import sqlite3
```

```
conn = sqlite3.connect("school.db")

top = pd.read_sql_query("SELECT name, marks FROM students WHERE marks > 60",
conn)

top.to_sql("top_students", conn, if_exists="replace", index=False)   # write
a table
conn.close()
```



sources, DataFrame and outputs

The same DataFrame methods work no matter where the data came from, which is why loading is the first and simplest step.

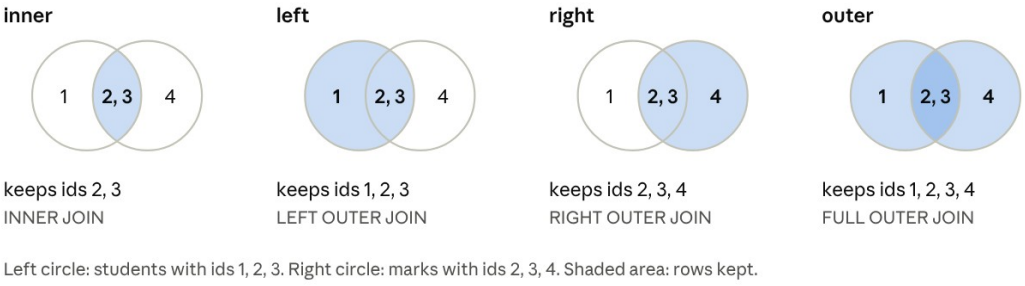
2. Combining and Merging Data Sets

Data usually arrives in several pieces; pandas combines them by matching keys (`merge`), by gluing along an axis (`concat`) or by filling gaps from another object (`combine_first`).

2.1 Database-style joins with `merge`

`pd.merge` links rows of two DataFrames that share a key, exactly like a SQL join. The `how` argument decides what happens to rows whose key appears in only one table.

`how`	Rows kept	SQL equivalent
`"inner"` (default)	Keys found in **both** tables	`INNER JOIN`
`"left"`	All keys of the left table; missing right values become `NaN`	`LEFT OUTER JOIN`
`"right"`	All keys of the right table	`RIGHT OUTER JOIN`
`"outer"`	Keys found in **either** table	`FULL OUTER JOIN`



merge join types · 4 Venn pictures

```
students = pd.DataFrame({"id": [1, 2, 3], "name": ["Asha", "Ravi", "Meena"]})
marks    = pd.DataFrame({"id": [2, 3, 4], "marks": [85, 91, 70]})

print(pd.merge(students, marks, on="id"))                # inner: ids 2 and
3
print(pd.merge(students, marks, on="id", how="left"))    # ids 1, 2, 3 (Asha
has NaN marks)
print(pd.merge(students, marks, on="id", how="outer"))   # ids 1, 2, 3, 4

# explicit key columns, and suffixes for clashing column names
pd.merge(students, marks, left_on="id", right_on="id", suffixes=("_s", "_m"))

# merge on the row labels instead of a column
pd.merge(students.set_index("id"), marks.set_index("id"), left_index=True,
right_index=True)
```

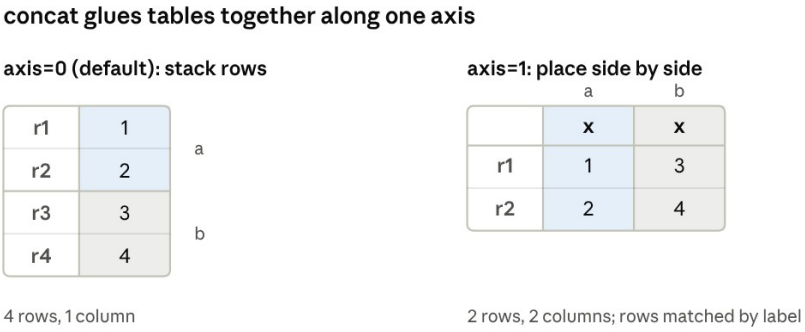
Useful arguments are `on` (key column or list of columns), `left_on` and `right_on` (different key names), `left_index` and `right_index` (use the row labels), `suffixes` (names for clashing columns) and `indicator=True` (adds a column showing which table each row came from). `DataFrame.join` is a shortcut for merging on the row labels. If a key repeats on both sides, the result has every pairing (many-to-many), so check for duplicates first.

2.2 Concatenating along an axis

`pd.concat` glues objects together instead of matching keys. `axis=0` (default) stacks them vertically, adding rows; `axis=1` places them side by side, aligning on the row labels.

```
a = pd.DataFrame({"x": [1, 2]}, index=["r1", "r2"])
b = pd.DataFrame({"x": [3, 4]}, index=["r3", "r4"])

print(pd.concat([a, b]))                          # 4 rows: r1, r2, r3, r4
print(pd.concat([a, b], ignore_index=True))       # rows renumbered 0 to 3
print(pd.concat([a, b], keys=["first", "second"]))   # adds a hierarchical
index
print(pd.concat([a, b.rename(index={"r3": "r1", "r4": "r2"})], axis=1))   #
side by side
```



concat along axis 0 and axis 1

2.3 Combining data with overlap

`combine_first` takes values from the first object and fills its gaps (`NaN`) from the second, matching on labels. It patches one data set with another.

```
import numpy as np

primary = pd.Series([1.0, np.nan, 3.0], index=["a", "b", "c"])
backup  = pd.Series([10.0, 20.0, 30.0], index=["a", "b", "c"])
print(primary.combine_first(backup))      # a 1.0, b 20.0, c 3.0
```

3. Reshaping and Pivoting

Reshaping changes the layout of a table without changing its data, most often between a **wide** layout (one column per category) and a **long** layout (one row per observation).

3.1 Reshaping with hierarchical indexing

`stack()` rotates the columns into an inner index level (wide to long) and `unstack()` rotates an index level into columns (long to wide); Unit 3 explained these on a MultiIndex.

3.2 Pivoting from long to wide

```
long = pd.DataFrame({
    "student": ["Asha", "Asha", "Ravi", "Ravi"],
    "subject": ["Maths", "Physics", "Maths", "Physics"],
    "marks":   [72, 65, 85, 90],
})

wide = long.pivot(index="student", columns="subject", values="marks")
print(wide)
# subject  Maths  Physics
# student
# Asha        72       65
# Ravi        85       90
```

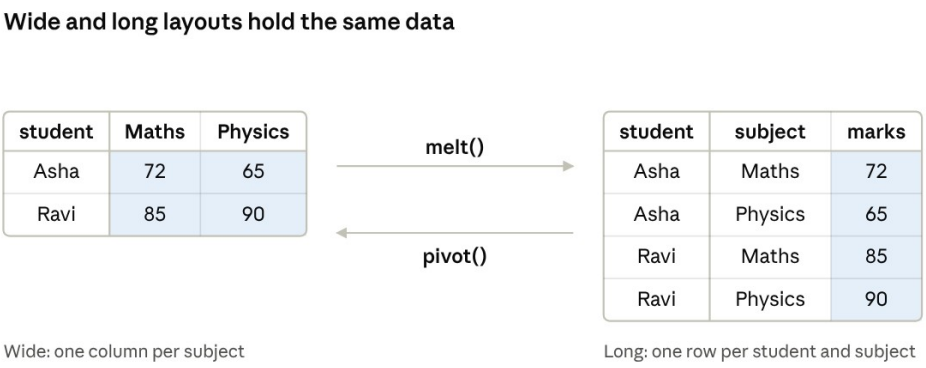
`pivot` needs each index and column pair to occur only once; if pairs repeat, use `pivot_table` (Section 6), which aggregates them.

3.3 Pivoting from wide to long

`pd.melt` does the reverse: it keeps the identifier columns and turns the other columns into rows of a *variable* name and a *value*.

```
back = wide.reset_index().melt(id_vars="student",
                               var_name="subject", value_name="marks")
print(back)
#   student  subject  marks
# 0    Asha    Maths     72
# 1    Ravi    Maths     85
# 2    Asha  Physics     65
# 3    Ravi  Physics     90
```

Layout	Shape	Best for
Wide	One row per student, one column per subject	Reading by eye, spreadsheets, reports
Long	One row per student and subject pair	`groupby`, plotting libraries and databases



wide and long layouts · melt and pivot

4. Data Transformation

Transformation fixes and reshapes the values themselves: removing duplicates, replacing codes, grouping numbers into bins, handling outliers and encoding categories as numbers.

4.1 Removing duplicates

```
df = pd.DataFrame({"name": ["Asha", "Ravi", "Asha", "Ravi"],
                   "dept": ["CSE", "IT", "CSE", "ECE"]})

print(df.duplicated())                      # False False True False  (marks
repeat rows)
print(df.drop_duplicates())                 # keeps the first of each
repeated row
print(df.drop_duplicates(subset=["name"]))   # compare by name only
print(df.drop_duplicates(subset=["name"], keep="last"))   # keep the last one
instead
```

4.2 Transforming data using a function or mapping

```
students = pd.DataFrame({"name": ["asha", "RAVI"], "dept": ["CSE", "IT"]})

students["name"] = students["name"].str.title()                    # Asha,
Ravi
students["school"] = students["dept"].map({"CSE": "Computing", "IT":
"Information"})
students = students.rename(columns={"dept": "department"})         #
rename columns
students.index = students.index.map(lambda i: "S" + str(i))        #
rename row labels
```

4.3 Replacing values

Real files use codes such as `-999` for "no data". `replace` swaps them for proper missing values.

```
import numpy as np

s = pd.Series([1, -999, 2, -999, 3])
print(s.replace(-999, np.nan))              # 1.0, NaN, 2.0, NaN, 3.0
print(s.replace([-999, 3], [0, 30]))        # several replacements at once
print(s.replace({-999: np.nan, 3: 30}))     # same, written as a dictionary
```

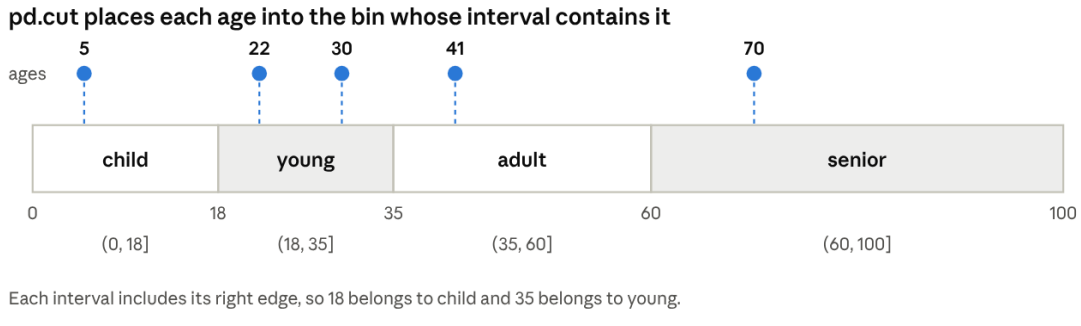
4.4 Discretization and binning

`pd.cut` sorts continuous values into bins you choose; `pd.qcut` makes bins that hold roughly equal numbers of values (quantiles). Each bin includes its right edge by default.

```
ages = [5, 22, 30, 41, 70]
bins = [0, 18, 35, 60, 100]                # intervals (0,18], (18,35],
(35,60], (60,100]
labels = ["child", "young", "adult", "senior"]

groups = pd.cut(ages, bins=bins, labels=labels)
print(list(groups))                         # ['child', 'young', 'young',
'adult', 'senior']
print(pd.Series(groups).value_counts())     # young 2, child 1, adult 1,
senior 1

print(pd.qcut(range(100), 4).value_counts()) # four bins of 25 values each
```



age bins · 4 intervals, 5 values

4.5 Detecting and filtering outliers

An outlier is a value far from the rest. A common rule flags values more than 1.5 times the interquartile range (IQR) beyond the quartiles.

```
x = pd.Series([10, 12, 11, 13, 12, 95])

q1, q3 = x.quantile(0.25), x.quantile(0.75)
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr

print(x[(x < low) | (x > high)])             # 95 is flagged as an outlier
print(x[(x >= low) & (x <= high)])           # remove the outlier
print(x.clip(lower=low, upper=high))         # or cap it at the limit
```

4.6 Dummy variables and scaling

Machine-learning models need numbers, so a category column is converted into one 0/1 **indicator** column per category with `get_dummies`. Numeric columns are often rescaled to a common range.

```
df = pd.DataFrame({"dept": ["CSE", "IT", "CSE"], "marks": [72, 85, 91]})

print(pd.get_dummies(df["dept"], prefix="dept", dtype=int))
#    dept_CSE  dept_IT
# 0         1        0
# 1         0        1
# 2         1        0

m = df["marks"]
minmax = (m - m.min()) / (m.max() - m.min())   # min-max scaling to the range 0 to 1
zscore = (m - m.mean()) / m.std()              # standardising: mean 0, standard deviation 1
```

To shuffle or sample rows use `df.sample(n=2)` or `df.sample(frac=1)`; `np.random.permutation(len(df))` gives a random ordering of positions.

5. String Manipulation

Text columns need cleaning too: Python's string methods, regular expressions and pandas' vectorized `.str` methods cover almost every case.

5.1 String object methods

Method	Purpose	Example
`split(sep)`	Break into a list	`"a,b,c".split(",")` gives `['a', 'b', 'c']`
`strip()`	Remove whitespace at both ends	`"  hi ".strip()` gives `'hi'`
`"sep".join(list)`	Glue a list into one string	`"-".join(["a", "b"])` gives `'a-b'`

Method	Purpose	Example
`replace(old, new)`	Substitute text	`"2026".replace("6", "7")` gives `'2027'`
`find(sub)` / `index(sub)`	Position of a match (`find` gives -1 if none, `index` raises an error)	`"data".find("t")` gives 2
`count(sub)`	Number of occurrences	`"banana".count("a")` gives 3
`startswith` / `endswith`	Test the ends	`"file.csv".endswith(".csv")` gives `True`
`lower()` / `upper()` / `title()`	Change case	`"asha".title()` gives `'Asha'`

5.2 Regular expressions

A **regular expression** (regex) is a pattern that describes a set of strings. Python's `re` module searches, splits and replaces with such patterns.

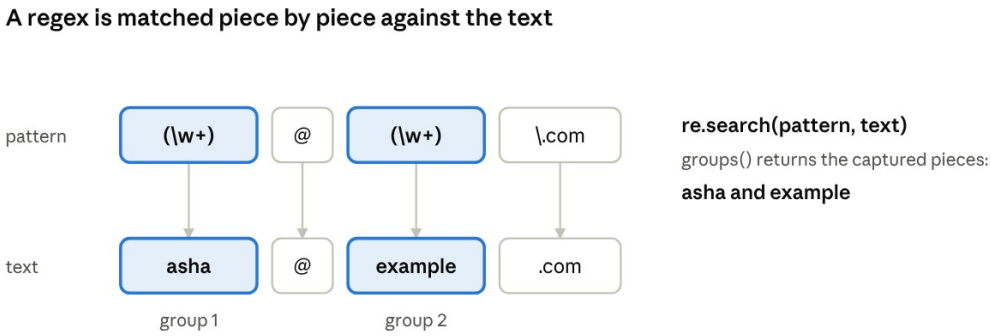
Pattern	Matches
`\d` / `\D`	A digit / a non-digit
`\w` / `\W`	A letter, digit or underscore / anything else
`\s`	Whitespace
`.`	Any one character except a newline
`*`, `+`, `?`	Zero or more, one or more, zero or one of the previous item
`{n}`, `{n,m}`	Exactly n, or n to m, of the previous item
`[abc]`, `[^abc]`	One of a, b, c / any character except a, b, c
`^`, `$`	Start / end of the string
`( )`	A group, captured for reuse

```
import re

text = "Call 98765 43210 or email asha@example.com by 2026-10-09"

print(re.findall(r"\d{5}", text))                          # ['98765',
'43210']
print(re.search(r"([\w.]+)@([\w.]+)", text).groups())      # ('asha',
'example.com')
print(re.sub(r"\s+", " ", "a   b \t c"))                  # 'a b c'
print(re.split(r"[,;]\s*", "x, y;z"))                     # ['x', 'y', 'z']
```

```
pattern = re.compile(r"(\d{4})-(\d{2})-(\d{2})")           # compile once,
reuse many times
print(pattern.search(text).groups())                        # ('2026', '10',
'09')
```



regex pattern matched against text

Use a raw string (`r"..."`) for patterns so that backslashes reach the regex engine unchanged. `match` tests only the start of a string, while `search` looks anywhere.

5.3 Vectorized string functions in pandas

The `.str` accessor applies a string operation to every item of a Series and skips missing values, so you need no loop.

```
emails = pd.Series(["Asha@Example.com", "ravi@test.org", None])

print(emails.str.lower().tolist())              # ['asha@example.com',
'ravi@test.org', nan]
print(emails.str.contains("example", case=False))   # True, False, NaN
print(emails.str.extract(r"@(\w+)\."))          # domain names: Example,
test, NaN
print(emails.str.split("@", expand=True))       # two columns: user and
domain
print(emails.str.len())                         # 16.0, 13.0, NaN
```

Method	Purpose
`str.lower`, `str.upper`, `str.title`, `str.strip`	Clean and change case
`str.contains(pat)`	True where the pattern is found; use it to filter rows
`str.replace(pat, repl)`	Substitute text (use `regex=True` for patterns)
`str.extract(pat)`	Pull the captured groups of a regex into columns
`str.split(sep, expand=True)`	Split into separate columns
`str.startswith`, `str.endswith`, `str.len`	Test the ends, count characters

6. Data Aggregation and Group Operations

Summarising data by category is done with the **split-apply-combine** pattern: split the rows into groups by a key, apply a calculation to each group, and combine the results into one object.

All examples in this section use this DataFrame.

```
import pandas as pd

df = pd.DataFrame({
    "dept":   ["CSE", "CSE", "IT", "IT", "IT"],
    "gender": ["F", "M", "F", "M", "F"],
    "marks":  [72, 91, 85, 64, 78],
})
```

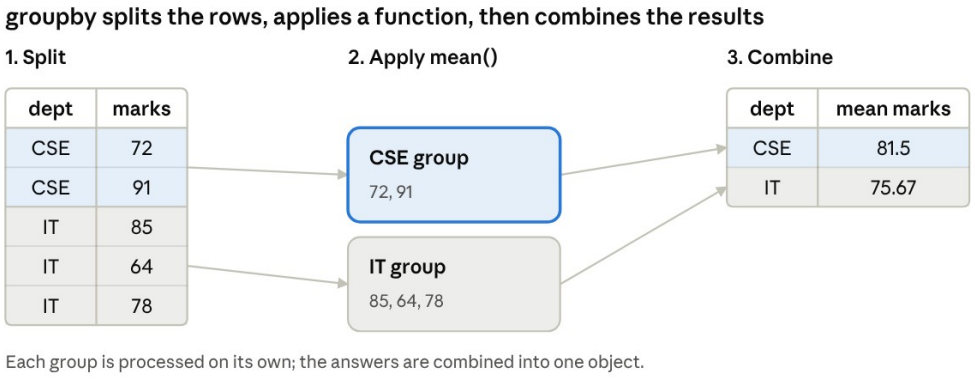
6.1 GroupBy mechanics

`df.groupby(key)` does not calculate anything yet; it builds a GroupBy object that remembers the groups. Choosing a column and a function then triggers the work.

```
g = df.groupby("dept")

print(g["marks"].mean())      # CSE 81.5, IT 75.666667
print(g.size())               # CSE 2, IT 3     rows per group
print(df.groupby(["dept", "gender"])["marks"].mean())   # one group per pair
of keys
# CSE F 72.0, CSE M 91.0, IT F 81.5, IT M 64.0

for name, group in g:         # iterate over the groups
    print(name, len(group))   # CSE 2, then IT 3
```



split-apply-combine · 3 stages

You can group by one column, a list of columns, or even a function or dictionary applied to the index. The result of a one-key grouping is indexed by the group names.

6.2 Data aggregation

An **aggregation** reduces each group to a single value. Common choices are `sum`, `mean`, `median`, `min`, `max`, `count`, `std`, `size`, `first` and `last`.

```
print(g["marks"].agg(["min", "max", "mean"]))      # several functions at
once

print(g.agg(avg=("marks", "mean"),                 # named aggregation: new
column = (column, function)
           top=("marks", "max")))

spread = lambda s: s.max() - s.min()               # your own function
print(g["marks"].agg(spread))                      # CSE 19, IT 21
```

6.3 Group operations and transformations

- **`transform`** returns a result with the **same shape as the input**, so it can be added back as a column.
- **`apply`** runs any function on each group and combines the pieces; it is the most flexible tool.
- **`filter`** keeps or drops whole groups by a condition.
- **`rank`**, **`cumsum`**, **`shift`** can also work within each group.

```
df["dept_mean"] = g["marks"].transform("mean")        # each row gets its
group average
df["above_avg"] = df["marks"] - df["dept_mean"]        # centre each mark on
its group

big = df.groupby("dept").filter(lambda d: len(d) >= 3)   # keep only
departments with 3+ rows

top1 = (df.sort_values("marks", ascending=False)
          .groupby("dept").head(1))                    # the best row of each
department
print(top1[["dept", "marks"]])                         # CSE 91, IT 85
```

Method	Output shape	Typical use
`agg`	One row per group	Totals, averages, counts per category
`transform`	Same length as the input	Group-wise scaling, filling gaps with group means
`apply`	Anything	Custom per-group logic, such as the top n rows
`filter`	A subset of the original rows	Remove small or unwanted groups

6.4 Pivot tables and cross tabulation

A **pivot table** is a groupby whose result is arranged as a two-way table: one key down the side, another across the top.

```
print(pd.pivot_table(df, values="marks", index="dept", columns="gender",
                     aggfunc="mean", margins=True))
# gender      F     M        All
# dept
```

```
# CSE     72.00  91.0  81.500000
# IT      81.50  64.0  75.666667
# All     78.33  77.5  78.000000
```
↗ building a pivot table

`margins=True` adds the "All" row and column, `fill_value=0` replaces empty cells and `aggfunc` can be a list such as `["mean", "count"]`.

A **cross tabulation** is a special pivot table that counts how often each pair of values occurs.

```
print(pd.crosstab(df["dept"], df["gender"]))
# gender  F  M
# dept
# CSE     1  1
# IT      2  1

print(pd.crosstab(df["dept"], df["gender"], normalize="index"))   # each row sums to 1
```

Tool	Best for
`groupby` + `agg`	A flexible summary with several keys and several functions
`pivot_table`	A summary shown as a two-way table
`crosstab`	Counting combinations of two categorical columns

Quick revision

Remember one core idea for each topic.

- **Text formats:** `read_csv` (with `sep`, `header`, `index_col`, `na_values`, `parse_dates`, `chunksize`) and `to_csv(index=False)`.
- **Binary formats:** pickle, HDF5 and Parquet keep types and load faster; load pickles only from trusted sources.
- **HTML and web APIs:** `read_html` returns a list of DataFrames; `requests.get(...).json()` then `pd.DataFrame(...)` for an API.
- **Databases:** `read_sql_query` to load a query result, `to_sql` to save a table.
- **merge:** a SQL-style join on keys; `how` is inner, left, right or outer.
- **concat:** stacks rows (`axis=0`) or places objects side by side (`axis=1`); `combine_first` fills gaps from another object.
- **Pivoting:** `pivot` goes long to wide, `melt` goes wide to long, `stack` and `unstack` move index levels.
- **Duplicates and replacement:** `drop_duplicates`, `replace(-999, np.nan)`, `map`, `rename`.
- **Binning:** `cut` uses bin edges you choose; `qcut` uses quantiles.
- **Outliers and dummies:** flag values beyond 1.5 times the IQR; `get_dummies` creates one indicator column per category.
- **Strings:** string methods, `re` patterns, and the vectorized `.str` accessor in pandas.

- **GroupBy:** split, apply, combine; `agg` reduces, `transform` keeps the shape, `apply` is flexible, `filter` drops groups.
- **Pivot tables and crosstab:** `pivot_table` summarises with an aggregation function; `crosstab` counts combinations.

Likely exam questions

Short answer (2 to 3 marks)

1. List any four options of `read_csv` and state their use.
2. Differentiate between `merge` and `concat`.
3. What is the difference between the inner and outer join?
4. What is the difference between `pivot` and `melt`?
5. Differentiate between `cut` and `qcut`.
6. What is a dummy variable? How is it created?
7. What is the split-apply-combine strategy?
8. Differentiate between `pivot_table` and `crosstab`.

Long answer (5 to 10 marks)

1. Explain how data is read from and written to text files, binary files, web pages and databases using pandas.
2. Explain the different ways of combining and merging data sets, with examples.
3. Explain reshaping and pivoting, with an example of converting between long and wide formats.
4. Explain the data transformation methods: removing duplicates, replacing values, binning and outlier detection.
5. Explain string manipulation with string methods, regular expressions and the pandas `.str` functions.
6. Explain GroupBy mechanics with aggregation and transformation examples.
7. Explain pivot tables and cross tabulation with examples.

References

Text books

1. Mark Lutz, *Learning Python*, O'Reilly.
2. Mark Lutz, *Programming Python*, O'Reilly.
3. Wes McKinney, *Python for Data Analysis*, O'Reilly (data loading and file formats, data wrangling, data aggregation and group operations).

Reference books

1. Martin C. Brown, *Python: The Complete Reference*, McGraw Hill Education.
2. Paul Barry, *Head First Python*, O'Reilly.